This version: 2017/08/12
Revising my paper before submitting for publication.

Syntactic Conditions for Antichain Property in CR-Prolog

Vu Phan
Texas Tech University

Abstract

We study syntactic conditions which guarantee when a Consistency Restoring Prolog program has antichain property (no answer set is a proper subset of another). A notable such condition is that the program's dependency graph being acyclic and having no directed path from one cr-rule head literal to another.

Contents

1 Introduction **1**

2 CR-Prolog **2**
2.1 Syntax . . . 2
2.2 Semantics . . . 2

3 Preliminary Results **3**
3.1 Antichain Sufficient Condition: One CR-Rule . . . 4
3.2 Antichain Sufficient Condition: Unique CR-Rule Head Literals . . . 4
3.3 Absence of Antichain Property: Necessary Conditions . . . 6

4 Main Results **7**
4.1 Dependency Graphs . . . 7
4.2 Antichain Sufficient Condition: Same Abductive Support Heads . . . 8
4.3 Antichain Sufficient Condition: CR-Independence and Acyclicity . . . 10

5 Conclusion **13**

6 Acknowledgment **13**

1 Introduction

- Answer Set Prolog (A-Prolog) is a programming language for knowledge representation and reasoning (Gelfond and Lifschitz [1988]). An A-Prolog program comprises rules which determine the sets of beliefs that a logical agent can hold. A-Prolog relies on the stable model semantics of logic programs with negation.

- Consistency Restoring Prolog (CR-Prolog) extends A-Prolog with cr-rules (Balduccini and Gelfond [2003]). The cr-rules apply only when the regular rules alone result in a contradiction. These cr-rules are meant to represent exceptions which are very rare but still possible.

- A-Prolog and its extension CR-Prolog have been applied to solve problems in various fields (Erdem et al. [2016]). For instance, a logic program was used to guide multiple robots to collaboratively tidy up a house. Also, a tourism application suggested trips based on user preferences.

- In this paper, we investigate the antichain property that a logic program might have: no answer set is a proper subset of another. Every A-Prolog program has antichain property. However, there exist CR-Prolog programs without antichain property. We look at some syntactic conditions guaranteeing that a CR-Prolog program has this desired semantic property. Such conditions are specified in Theorem 4.3.12 (most notable) as well as Theorem 3.1.3, Corollary 3.1.6, and Propositions 3.2.10 & 4.2.11.

2 CR-Prolog

The complete specifications of A-Prolog and CR-Prolog can be found in [Gelfond and Kahl, 2014, Sections 2.1 & 2.2 & 5.5].

2.1 Syntax

Definition 2.1.1. An **atom** represents a boolean value. A **literal** is either an atom a or its **classical-negation** $\neg a$. An **extended literal** is either a literal l or its **default-negation** $not\ l$.

Definition 2.1.2 (Regular Rule)**.** A **regular rule** r has the form:

$$l_1\ or \ldots or\ l_k \longleftarrow l_{k+1}, \ldots, l_m,\ not\ l_{m+1}, \ldots, not\ l_n.$$

Each l_i above is a literal and $1 \leq k \leq m \leq n$. If $k = m = n$, then we call r a **fact** and write:

$$l_1\ or \ldots or\ l_k.$$

The **head** of the rule r is the disjunction of literals before the arrow "$\longleftarrow$". The **body** of r is the conjunction of extended literals after "$\longleftarrow$".

Definition 2.1.3 (CR-Rule)**.** Similar to a regular rule, a **cr-rule** has the form:

$$l_0 \overset{+}{\longleftarrow} l_1, \ldots, l_m,\ not\ l_{m+1}, \ldots, not\ l_n.$$

Definition 2.1.4 (A-Prolog Program)**.** An **A-Prolog program** is a finite set of regular rules.

Definition 2.1.5 (CR-Prolog Program)**.** A **CR-Prolog program** is a finite set of regular rules and cr-rules. Let Π^{reg} and Π^{cr} denote the corresponding subsets of regular rules and cr-rules.

Definition 2.1.6 (Applications of CR-Rules)**.** Let r be a cr-rule. The **application** $\alpha(r)$ is the regular rule obtained from r by replacing "$\overset{+}{\longleftarrow}$" with "$\longleftarrow$". If R is a set of cr-rules, then let $\alpha(R) = \{\alpha(r) : r \in R\}$. When $R \subseteq \Pi^{cr}$ (meaning R is a subset of cr-rules in a CR-Prolog program Π), define $\Pi_R = \Pi^{reg} \cup \alpha(R)$.

2.2 Semantics

Definition 2.2.1 (Context)**.** Literals l and l' are **complementary** if $l' = \neg l$ (one is the classical-negation of the other). A set S of literals is a **context** if S contains no pair of complementary literals. Note: for simplicity, we have slightly altered the original definition of a "context" in [Ben-Eliyahu and Dechter, 1994, page 55].

Definition 2.2.2 (Satisfiability)**.** A context S satisfies:

1. a literal l if $l \in S$
2. an extended literal $not\ l$ if $l \notin S$
3. a disjunction $l_1\ or \ldots or\ l_k$ if some $l_i \in S$

4. a conjunction $l_{k+1}, \ldots, l_m,\ not\ l_{m+1}, \ldots, not\ l_n$ if S satisfies all conjuncts (which are extended literals)

5. a rule r if S satisfies the head of r whenever S satisfies the body of r

6. a program Π if S satisfies every rule in Π

Definition 2.2.3 (A-Prolog Answer Set, part 1 of 2)**.** Let Π be an A-Prolog program without default-negation. Then a context S is an answer set of Π if:

1. S satisfies Π, and

2. S is minimal (wrt set-inclusion): no proper subset of S satisfies Π

Definition 2.2.4 (A-Prolog Answer Set, part 2 of 2)**.** Let Π be an A-Prolog program and S be a context. The **reduct** Π^S is the program obtained from Π by:

1. removing each rule containing *not* l such that $l \in S$

2. removing all other extended literals containing *not*

Then S is an answer set of Π if S is an answer set of Π^S. If Π has some answer set, then Π is said to be **consistent**.

Definition 2.2.5 (Abductive Support)**.** Let Π be a CR-Prolog program and $R \subseteq \Pi^{cr}$ (meaning R is a subset of cr-rules in Π). Then R is an **abductive support** of Π if:

1. the A-Prolog program $\Pi^{reg} \cup \alpha(R)$ is consistent, and

2. R is minimal (wrt cardinality): no $R' \subseteq \Pi^{cr}$ exists such that $\Pi^{reg} \cup \alpha(R')$ is consistent where $|R'| < |R|$

Definition 2.2.6 (CR-Prolog Answer Set)**.** Let Π be a CR-Prolog program and S be a context. Then S is an answer set of Π if S is an answer set of $\Pi^{reg} \cup \alpha(R)$ for some abductive support R of Π.

Definition 2.2.7 (Antichain Property)**.** Let Π be a CR-Prolog program. We say Π has **antichain property** if: for all answer sets S_1 and S_2 of Π, we have $S_1 \subseteq S_2 \Rightarrow S_1 = S_2$.

Example 2.2.8 (CR-Prolog Program without Antichain Property)**.** Consider the following program:

$$
\begin{aligned}
a.& \\
\neg a \longleftarrow\ & not\ b, not\ c. \\
b \longleftarrow\ & c. \\
b \xleftarrow{+}\ & . && (r_1) \\
c \xleftarrow{+}\ & . && (r_2)
\end{aligned}
$$

This program has answer sets $S_1 = \{a, b\}$ and $S_2 = \{a, b, c\}$. Observe that $S_1 \subset S_2$ (proper subset). The corresponding abductive supports are $R_1 = \{r_1\}$ and $R_2 = \{r_2\}$.

3 Preliminary Results

This section provides important lemmas.

3.1 Antichain Sufficient Condition: One CR-Rule

Remark 3.1.1. Every inconsistent CR-Prolog program vacuously has antichain property. So only consistent programs are of interest.

Lemma 3.1.2. Let Π be an A-Prolog program and S_1 & S_2 be contexts. If $S_1 \subseteq S_2$, then $\Pi^{S_2} \subseteq \Pi^{S_1}$ (one reduct is a subset of the other).

Proof. Assume Π^{S_2} has an arbitrary rule r:

$$l_1 \ or \ldots or \ l_k \longleftarrow l_{k+1}, \ldots, l_m.$$

The corresponding rule in Π is:

$$l_1 \ or \ldots or \ l_k \longleftarrow l_{k+1}, \ldots, l_m, \ not \ l_{m+1}, \ldots, not \ l_n.$$

For each i in $\{m+1, \ldots, n\}$, we have $l_i \notin S_2$, so $l_i \notin S_1$ (as $S_1 \subseteq S_2$). Therefore, r is also a rule in Π^{S_1}. □

Theorem 3.1.3 (Antichain Property of A-Prolog Programs, proved by Dr. Michael Gelfond)**.** Let Π be an A-Prolog program and $S_1 \subseteq S_2$ be answer sets of Π. Then $S_1 = S_2$.

Proof. We will show $S_2 \subseteq S_1$.

1. First, S_2 is an answer set of Π^{S_2}. So S_2 is a minimal (wrt set-inclusion) context that satisfies Π^{S_2}. We shall prove $S_2 \subseteq S_1$ by showing S_1 also satisfies Π^{S_2}.

2. By 3.1.2, we have $\Pi^{S_2} \subseteq \Pi^{S_1}$. Because S_1 satisfies Π^{S_1}, we know S_1 also satisfies Π^{S_2}.

□

Definition 3.1.4 (Equivalence of CR-Prolog Programs)**.** Let Π_1 and Π_2 be CR-Prolog programs. We say Π_1 and Π_2 are **equivalent** when: S is an answer set of Π_1 iff S is an answer set of Π_2, for every context S.

Remark 3.1.5. Let Π be a CR-Prolog program. If Π^{reg} is consistent, then Π is equivalent to Π^{reg}, which has antichain property due to Theorem 3.1.3. The interesting case is when Π^{reg} is inconsistent.

Corollary 3.1.6. Let Π be a CR-Prolog program where Π^{reg} is inconsistent. If Π has only one cr-rule, then Π has antichain property.

Proof. Assume $\Pi^{cr} = \{r\}$ for some cr-rule r. Then Π is equivalent to the A-Prolog program $\Pi_{\{r\}} = \Pi^{reg} \cup \alpha(\{r\})$. Simply apply Theorem 3.1.3 to $\Pi_{\{r\}}$. □

3.2 Antichain Sufficient Condition: Unique CR-Rule Head Literals

Definition 3.2.1 (Useless Regular Rule)**.** Let Π be an A-Prolog program containing some rule r. For an answer set S of Π, we say r is **useless** wrt S if S is an answer set of $\Pi \setminus \{r\}$.

Definition 3.2.2 (Regular Rule Firing)**.** Given a regular rule r and a context S, we say r **fires** wrt S if S satisfies the body of r.

Lemma 3.2.3. Let r be a regular rule and S be a context. Assume the reduct $\{r\}^S = \{r'\}$ for some default-negation-free rule r'. Then r fires iff r' fires, wrt S.

Proof. These two rules have the forms:

$$l_1 \ or \ldots or \ l_k \longleftarrow l_{k+1}, \ldots, l_m, \ not \ l_{m+1}, \ldots, not \ l_n. \tag{r}$$

$$l_1 \ or \ldots or \ l_k \longleftarrow l_{k+1}, \ldots, l_m. \tag{r'}$$

By construction of r' from r, we know $l_i \notin S$ for all $i \in \{m+1, \ldots, n\}$. Then r fires wrt S iff $l_i \in S$ for all $i \in \{k+1, \ldots, m\}$ iff r' fires wrt S. □

Remark 3.2.4. Let r be a regular default-negation-free rule and $S_1 \subseteq S_2$ be contexts. If r fires wrt S_1, then r fires wrt S_2.

Lemma 3.2.5. Let Π be an A-Prolog program containing some rule r and S be an answer set of Π. If r does not fire, then r is useless, wrt S.

Proof. Define the reducts $\Pi_1 = \Pi^S$ and $\Pi_2 = (\Pi \setminus \{r\})^S$. By Definition 2.2.4, S is an answer set of Π_1. We will prove S is an answer set of Π_2.

1. First, S satisfies Π_1. Notice $\Pi_2 \subseteq \Pi_1$. Then S satisfies Π_2.
2. Assume some context $S' \subseteq S$ also satisfies Π_2. We shall show $S' = S$.
   2.1. Case 1 of 2: $\{r\}^S = \varnothing$. Then $\Pi_1 = \Pi_2$, so S' satisfies Π_1.
   2.2. Case 2 of 2: $\{r\}^S = \{r'\}$ for some default-negation-free rule r'. Recall that r does not fire wrt S. By Lemma 3.2.3, r' does not fire wrt S. Since $S' \subseteq S$, using the contrapositive of Remark 3.2.4, we deduce that r' does not fire wrt S'. Then S' vacuously satisfies r'. So S' satisfies $\Pi_1 = \Pi_2 \cup \{r'\}$.

   In both cases, S' satisfies Π_1. As S is an answer set of Π_1, due to minimality, we know $S \subseteq S'$. Since we assumed $S' \subseteq S$, we can now conclude that $S' = S$.
3. We have shown that S is a minimal context which satisfies Π_2. Hence, S is an answer set of Π_2.

□

Lemma 3.2.6. Choose an arbitrary CR-Prolog program Π. Let S be an answer set of Π and R be a corresponding abductive support, meaning S is an answer set of the A-Prolog program $\Pi_R = \Pi^{reg} \cup \alpha(R)$. If cr-rule $r \in R$, then $\alpha(r)$ is not useless wrt S.

Proof. By way of contradiction, assume $\alpha(r)$ is useless wrt S. Then S is an answer set of $\Pi_R \setminus \{\alpha(r)\}$, whereas this program is precisely $\Pi_{R'} = \Pi^{reg} \cup \alpha(R')$ with $R' = R \setminus \{r\}$. So $\Pi_{R'}$ is consistent. By Definition 2.2.5, R is a minimal (wrt cardinality) set of cr-rules whose applications make Π consistent, we have $|R| \leq |R'|$. However, $|R'| = |R| - 1$ (recall $R' = R \setminus \{r\}$), contradiction. □

Lemma 3.2.7. Let Π be a CR-Prolog program with some answer set S and a corresponding abductive support R. If cr-rule $r \in R$, then the application $\alpha(r)$ fires wrt S.

Proof. First, S is an answer set of $\Pi_R = \Pi^{reg} \cup \alpha(R)$. By way of contradiction, assume there exists a cr-rule $r \in R$ such that $\alpha(r)$ does not fire wrt S. By Lemma 3.2.5, $\alpha(r)$ is useless wrt S. However, by Lemma 3.2.6, $\alpha(r)$ is not useless wrt S (as $r \in R$), contradiction. □

Definition 3.2.8 (Head)**.** If r is a rule, then let $head(r)$ denotes the set of literals which appear in the head of r. If R is a set of rules, then let $head(R) = \bigcup_{r \in R} head(r)$.

Definition 3.2.9 (Supported Literal)**.** Let r be a regular rule, l be a literal, and S be a context. We say l is **supported** by r wrt S if:

1. r fires wrt S, and
2. $head(r) \cap S = \{l\}$

[Gelfond and Kahl, 2014, page 43, Proposition 2.2.1]

Proposition 3.2.10. Let Π be a CR-Prolog program. If the literal in the head of each cr-rule does not appear in the head of any other rule, then Π has antichain property.

Proof. Let S_1 and S_2 be answer sets of Π. Assume $S_1 \subseteq S_2$. We will show $S_1 = S_2$.

1. Let R_1 and R_2 respectively be abductive supports for S_1 and S_2. We first prove $R_1 \subseteq R_2$. Choose an arbitrary cr-rule $r \in R_1$. By Lemma 3.2.7, r fires wrt S_1, and S_1 satisfies $head(r) = \{l\}$ where l is a literal. So $l \in S_1$, then $l \in S_2$ (as $S_1 \subseteq S_2$). By the hypothesized syntactic condition, r is the sole rule in Π whose head contains l. Then only r can support l wrt S_2 (in the sense of Definition 3.2.9). So $r \in R_2$.

2. Now that we have shown $R_1 \subseteq R_2$, recall $|R_1| = |R_2|$, as all abductive supports have the same minimal cardinality. Then $R_1 = R_2$, since they are finite sets. Consequently, $\Pi^{reg} \cup \alpha(R_1)$ is equal to $\Pi^{reg} \cup \alpha(R_2)$. So S_1 and S_2 are answer sets of the same A-Prolog program. By Theorem 3.1.3, $S_1 = S_2$.

□

3.3 Absence of Antichain Property: Necessary Conditions

Remark 3.3.1. If Π is a CR-Prolog program without antichain property, then:

1. Π is consistent (by Remark 3.1.1), and

2. Π^{reg} is inconsistent (by Remark 3.1.5), and

3. $|\Pi^{cr}| \geq 2$ (by Corollary 3.1.6)

Lemma 3.3.2. If a CR-Prolog program Π has some answer sets $S_1 \subset S_2$ (proper subset) with corresponding abductive supports R_1 and R_2, then $R_1 \neq R_2$.

Proof. By way of contradiction, assume $R_1 = R_2$. So $\Pi_{R_1} = \Pi^{reg} \cup \alpha(R_1)$ is identical to $\Pi_{R_2} = \Pi^{reg} \cup \alpha(R_2)$. Then S_1 and S_2 are answer sets of the same A-Prolog program $\Pi_{R_1} = \Pi_{R_2}$. By Theorem 3.1.3, $S_1 = S_2$, contradicting $S_1 \subset S_2$. □

Lemma 3.3.3. Let Π be a CR-Prolog program with some abductive support R and a corresponding answer set S. Then $head(R) \subseteq S$.

Proof. Let r be an arbitrary cr-rule in R. By Lemma 3.2.7, $\alpha(r)$ fires wrt S. So S satisfies $head(\alpha(r)) = head(r) = \{l\}$. Thus literal $l \in S$. □

Lemma 3.3.4. Let Π be an A-Prolog program and S_1 & S_2 be contexts. If S_1 satisfies Π, then S_1 satisfies Π^{S_2}.

Proof. In Π^{S_2}, choose an arbitrary default-negation-free rule r. Let the corresponding rule in Π be r'. Assume r fires wrt S_1. By Lemma 3.2.3, r' fires wrt S_1. As S_1 satisfies r', we know S_1 satisfies $head(r') = head(r)$. So S_1 satisfies r as well. □

Lemma 3.3.5. Let Π be an A-Prolog program with some answer set S. If a context S' satisfies Π, then $S' \subseteq S \Rightarrow S' = S$.

Proof. By Lemma 3.3.4, S' satisfies $\Pi^{S'}$. By Lemma 3.1.2, $\Pi^S \subseteq \Pi^{S'}$. Therefore, S' satisfies Π^S. Recall the answet set S is a minimal context which satisfies Π^S. Then $S \subseteq S'$. We assumed $S' \subseteq S$, so $S' = S$. □

Lemma 3.3.6 (Proved by Dr. Michael Gelfond)**.** If $S_1 \subset S_2$ are answer sets of a CR-Prolog program Π with some corresponding abductive supports R_1 & R_2, then:

1. $head(R_2) \not\subseteq S_1$, and

2. for each literal $l \in head(R_2) \setminus S_1$, there exists a rule $r \in R_2 \setminus R_1$ where $head(r) = \{l\}$.

Proof. We follow these steps.

1. First, we prove $head(R_2) \not\subseteq S_1$.

1.1. Assume otherwise that $head(R_2) \subseteq S_1$. Then S_1 trivially satisfies $\alpha(R_2)$.

1.2. As S_1 is an answer set of Π, we know S_1 satisfies Π^{reg}.

1.3. Therefore S_1 satisfies $\Pi_{R_2} = \Pi^{reg} \cup \alpha(R_2)$.

1.4. Recall S_2 is an answer set of Π_{S_2}.

1.5. By Lemma 3.3.5, $S_1 = S_2$. But $S_1 \subset S_2$, contradiction.

2. Now, let l be a literal in $head(R_2) \setminus S_1 \neq \varnothing$. Choose a cr-rule $r \in R_2$ such that $head(r) = \{l\}$. We show $r \notin R_1$.

2.1. By Lemma 3.3.3, $head(R_1) \subseteq S_1$.

2.2. If $r \in R_1$, then $\{l\} = head(r) \subseteq head(R_1) \subseteq S_1$. So $l \in S_1$, contradicting that $l \in head(R_2) \setminus S_1$. Therefore $r \notin R_1$.

□

4 Main Results

This section derives weaker syntactic conditions for antichain property.

4.1 Dependency Graphs

Definition 4.1.1 (Positive Body of Rule)**.** Let r be a rule. The **positive body** of r, denoted by $body_+(r)$, is the set of literals in the body of r which are default-negation-free.

Definition 4.1.2 (Dependency Graph)**.** Let Π be a CR-Prolog program. Let G_Π be a graph whose vertices are literals in Π. For each pair of vertices l_1 and l_2 of G_Π, there is a directed edge to l_1 from l_2 if Π has some rule r where $l_1 \in head(r)$ and $l_2 \in body_+(r)$. We say G_Π is the **dependency graph** of Π. Note: a self-loop forms a directed cycle; see Example 4.1.4.

Remark 4.1.3 (Answer Set of Acyclic Program)**.** Let Π be an acyclic A-Prolog program and S be a context. Then S is an answer set of Π iff:

1. S satisfies Π, and

2. every literal in S is supported by a rule in Π wrt S (in the sense of Definition 3.2.9)

[Ben-Eliyahu and Dechter, 1994, page 58, Theorem 2.7]

Example 4.1.4 (Self-Loop Forming Cycle)**.** To the contrary, assume a self-loop does not form a cycle. Consider this A-Prolog program containing the sole rule r:

$$l \longleftarrow l.$$

Let $S = \{l\}$. Notice that:

1. S satisfies Π, and

2. l is supported by r wrt S

However, S is not an answer set of Π. Then Remark 4.1.3 is wrong, contradiction.

Definition 4.1.5 (Head-Cycle)**.** Let Π be a CR-Prolog program with dependency graph G_Π. Let C be a directed cycle in G_Π. Assume:

1. vertices l_1 and l_2 are in C, and

2. there exists a rule $r \in \Pi$ where l_1 and l_2 are in $head(r)$

Then C is a **head-cycle** in Π [Ben-Eliyahu and Dechter, 1994, page 56].

Definition 4.1.6 (Dependent Literals)**.** Let l_1 and l_2 be two literals appearing in a CR-Prolog program Π. We say l_1 **depends** on l_2 if the dependency graph G_Π has a directed path to l_1 from l_2.

Definition 4.1.7 (CR-Literal)**.** Let Π be a CR-Prolog program with some cr-rule r where $head(r) = \{l\}$. We say l is a **cr-literal** in Π.

Definition 4.1.8 (CR-Independence)**.** Let Π be a CR-Prolog program. Then Π is called **cr-independent** if l_1 does not depend on l_2 for all cr-literals l_1 and l_2 in Π.

4.2 Antichain Sufficient Condition: Same Abductive Support Heads

Remark 4.2.1. Let Π be a CR-Prolog program with some answer set S. Then $S \subseteq head(\Pi)$.

Remark 4.2.2. Let r be a regular rule and S be a context. If r fires wrt S, then the reduct $\{r\}^S \neq \varnothing$.

Remark 4.2.3. Let Π be a CR-Prolog program with some answer set S and a corresponding abductive support R. Let cr-rule $r \in R$. Then the reduct $\{\alpha(r)\}^S \neq \varnothing$ (by Lemma 3.2.7 and Remark 4.2.2).

Lemma 4.2.4. Let Π be a nondisjunctive default-negation-free A-Prolog program. If contexts S_1 and S_2 satisfy Π, then $S_0 = S_1 \cap S_2$ also satisfies Π.

Proof. Let r be a rule in Π. Assume r fires wrt S_0. Then r also fires wrt S_1 and S_2 (as Π is default-negation-free). So S_1 and S_2 satisfy $head(r) = \{l\}$ (since Π is nondisjunctive). Thus l is in S_1, S_2, and S_0. □

Lemma 4.2.5 (Proved by Dr. Michael Gelfond)**.** Let Π be a nondisjunctive default-negation-free A-Prolog program. Assume Π has rules $r_1 \neq r_2$ such that $head(r_1) = head(r_2)$. Let $\Pi_0 = \Pi \setminus \{r_1, r_2\}$, $\Pi_1 = \Pi_0 \cup \{r_1\}$, and $\Pi_2 = \Pi_0 \cup \{r_2\}$. If S is an answer set of Π, then S is also an answer set of either Π_1 or Π_2.

Proof. To the contrary, assume S is an answer set of neither Π_1 nor Π_2. Still, S satisfies both Π_1 and Π_2 (as S satisfies Π). So there exist two proper subsets of S, say S_1 and S_2, which respectively satisfy Π_1 and Π_2 (the programs are default-negation-free).

1. Case 1 of 2: either r_1 fires wrt S_1, or r_2 fires wrt S_2. WOLOG, assume the former. Then S_1 satisfies $head(r_1) = head(r_2)$. So S_1 also satisfies both the rule r_2 and the program $\Pi = \Pi_1 \cup \{r_2\}$. Recall that S is an answer set of Π whereas $S_1 \subset S$ (proper subset), contradicting Definition 2.2.3.

2. Case 2 of 2: neither r_1 fires wrt S_1, nor r_2 fires wrt S_2. So neither r_1 nor r_2 fires wrt $S_0 = S_1 \cap S_2$ (the rules are default-negation-free). Then S_0 vacuously satisfies r_1 and r_2. Because S_1 and S_2 satisfy Π_0, by Lemma 4.2.4, S_0 satisfies Π_0 too. Therefore, S_0 satisfies $\Pi = \Pi_0 \cup \{r_1, r_2\}$. But S is an answer set of Π, and $S_0 \subset S$, contradiction.

□

Lemma 4.2.6. Let Π be a nondisjunctive CR-Prolog program with some abductive support R. For all cr-rules r_1 and r_2 in R, if $head(r_1) = head(r_2)$, then r_1 and r_2 are the same rule.

Proof. By way of contradiction, assume there exist cr-rules $r_1 \neq r_2$ in R where $head(r_1) = head(r_2)$. Some notations:

1. Let $R_1 = R \setminus \{r_2\}$ and $R_2 = R \setminus \{r_1\}$.

2. Let $\Pi_1 = \Pi^{reg} \cup \alpha(R_1)$ and $\Pi_2 = \Pi^{reg} \cup \alpha(R_2)$.

3. Let S be an answer set of $\Pi^{reg} \cup \alpha(R)$.

4. Let $\Pi_a = (\Pi_1)^S$ and $\Pi_b = (\Pi_2)^S$.

By Lemma 4.2.5, S is an answer set of either Π_a or Π_b. WOLOG, assume the former. Then S is an answer set of Π_1. So R_1 is another abductive support of Π, but $|R_1| < |R|$ (recall $R_1 = R \setminus \{r_2\}$), contradicting Definition 2.2.5. □

Lemma 4.2.7. Let Π be an acyclic CR-Prolog program having some answer set S with a corresponding abductive support R. Let cr-rule $r \in R$ where $head(r) = \{l\}$. Then $\alpha(r)$ is the only rule in $\Pi_R = \Pi^{reg} \cup \alpha(R)$ which supports literal l wrt S.

Proof. By way of contradiction, assume l is also supported by a rule $r' \neq \alpha(r)$ in Π_R. Let $R' = R \setminus \{r\}$ and $\Pi' = \Pi^{reg} \cup \alpha(R')$. We will prove S is an answer set of Π':

1. First, S satisfies $\Pi' \subseteq \Pi_R$.

2. Let l' be an arbitrary literal in S. We show l' is supported wrt S by some rule in Π'. Recall that S is an answer set of Π_R. Applying Remark 4.1.3 to Π_R, we know l' is supported wrt S by some rule r_0 in Π_R.

   2.1. Case 1 of 2: $r_0 = \alpha(r)$. Recall $head(r) = \{l\}$. Then $l = l'$. Notice r' also supports $l' = l$ wrt S, and $r' \in \Pi'$.

   2.2. Case 2 of 2: $r_0 \neq \alpha(r)$. Then $r_0 \in \Pi'$ by construction.

   In both cases, l' is supported by some rule in Π' wrt S.

Applying Remark 4.1.3 to Π', we deduce that S is an answer set of Π'. So Π' is consistent, and R' is an abductive support of Π. But $|R'| < |R|$, contradiction. □

Definition 4.2.8 (Factified Rule)**.** For a regular rule r, let $\phi(r)$ denote the **factified rule** obtained from r by dropping its body. If R is a set of rules, then define $\phi(R) = \{\phi(r) : r \in R\}$.

Lemma 4.2.9. Let Π be a CR-Prolog program with some answer set S and a corresponding abductive support R. Then S is also an answer set of the A-Prolog program $\Pi' = \Pi^{reg} \cup \phi(\alpha(R))$.

Proof. We prove S is a minimal context which satisfies the reduct $(\Pi')^S$:

1. Let $\Pi_R = \Pi^{reg} \cup \alpha(R)$. Recall S is an answer set of and thus satisfies $(\Pi_R)^S = (\Pi^{reg})^S \cup (\alpha(R))^S$. Since R is a corresponding abductive support, $head(R) \subseteq S$. Of course, $head(\phi(\alpha(R))) = head(\alpha(R)) = head(R)$. Then S satisfies $(\Pi')^S = (\Pi^{reg})^S \cup \phi(\alpha(R))$.

2. Assume some context $S' \subseteq S$ also satisfies $(\Pi')^S$. Since $\phi(\alpha(R))$ contains only facts, we know $head(\alpha(R)) = head(\phi(\alpha(R))) \subseteq S'$. Then S' satisfies $(\Pi_R)^S$. So $S \subseteq S'$ (recall S is an answer set of Π_R). Therefore $S' = S$.

□

Lemma 4.2.10. Let Π be a CR-Prolog program with answer sets $S_1 \subseteq S_2$ and corresponding abductive supports R_1 & R_2, respectively. If $head(R_1) = head(R_2)$, then $S_1 = S_2$.

Proof. By Lemma 4.2.9, S_1 and S_2 are respectively answer sets of $\Pi^{reg} \cup \phi(\alpha(R_1))$ and $\Pi^{reg} \cup \phi(\alpha(R_2))$, which are the same A-Prolog program because $head(R_1) = head(R_2)$. By Theorem 3.1.3, $S_1 \subseteq S_2 \Rightarrow S_1 = S_2$. □

Proposition 4.2.11. Let Π be a CR-Prolog program. Then Π has antichain property if $head(R_1) = head(R_2)$ for all abductive supports R_1 and R_2 of Π.

Proof. Let S_1 and S_2 be arbitrary answer sets of Π corresponding to some abductive supports R_1 and R_2, respectively. By Lemma 4.2.10, $S_1 \subseteq S_2 \Rightarrow S_1 = S_2$. □

4.3 Antichain Sufficient Condition: CR-Independence and Acyclicity

Definition 4.3.1 (Proof of Literal)**.** Let Π be an A-Prolog program, S be a context, and l be a literal. A **proof** of l wrt S in Π is a nonempty sequence of rules $p = \langle r_1, \ldots, r_n \rangle$ in Π such that:

1. for each rule r_i, there is a (sole) literal supported by r wrt S; we call this literal $h_S(r)$
2. $l = h_S(r_n)$
3. $body_+(r_1) = \varnothing$
4. for every rule r_i, each literal in $body_+(r_i)$ is $h_S(r_j)$ for some $j < i$

[Ben-Eliyahu and Dechter, 1994, page 57]

Note on point 3. Their original definition states: r_1 is a fact. However, there is an error in their Lemma B.5 on page 83. Consider the acyclic A-Prolog program $\{a \longleftarrow not\ b.\}$ with the answer set $\{a\}$.

Additional notations:

- $P(l, S, \Pi)$ denotes the set of all proofs p of l wrt S in Π; a proof $p \in P(l, S, \Pi)$ is called **minimal** if p is shortest (containing fewest rules)
- $h_S(p)$ denotes $\{h_S(r) : r \in p\}$
- $body_+(p)$ denotes $\{body_+(r) : r \in p\}$

Remark 4.3.2. Let $p = \langle r_1, \ldots, r_n \rangle$ be a proof in $P(l, S, \Pi)$; as usual, each r_i is a rule, l is a literal, S is a context, and Π is an A-Prolog program. We adopt the convention that the rules in p are pairwise distinct. Indeed, if there were rules $r_i = r_j$ where $i < j$, then $p' = \langle r_1, \ldots, r_{j-1}, r_{j+1}, \ldots, r_n \rangle$ would be a shorter proof in $P(l, S, \Pi)$, and r_j would be redundant.

Remark 4.3.3. Let Π be a head-cycle-free A-Prolog program with an answer set S. Then each literal in S has a proof wrt S in Π [Ben-Eliyahu and Dechter, 1994, page 57, Theorem 2.3].

Definition 4.3.4 (Rank of Literal)**.** Let Π be a head-cycle-free A-Prolog program, S be a context, and l be a literal. The **rank** $rank(l, S, \Pi)$ of l wrt S in Π is $\min\{|p| : p \in P(l, S, \Pi)\}$, which is the length of a shortest proof p of l wrt S in Π.

Definition 4.3.5 (Ranking Function)**.** Let:

1. Π be a head-cycle-free A-Prolog program
2. S be an answer set of Π
3. f be a function from S to the set of positive integers

Then f is called the **ranking function** wrt S in Π if $f(l) = rank(l, S, \Pi)$ for all $l \in S$.

Definition 4.3.6 (Normal Proof of Literal)**.** Let:

1. Π be a head-cycle-free A-Prolog program
2. S be an answer set of Π
3. l be a literal in S
4. $p \in P(l, S, \Pi)$; recall that this notation means p is a proof of l wrt S in Π

5. f be the ranking function wrt S in Π

We say p is a **normal proof** of l wrt S in Π if: for each rule $r \in p$ and each literal $l' \in body_+(r)$, we have $f(h_S(r)) > f(l')$. This definition is inspired by [Ben-Eliyahu and Dechter, 1994, page 59, Theorem 2.8].

Remark 4.3.7. Let:

1. Π be a head-cycle-free A-Prolog program
2. S be an answer set of Π
3. f be the ranking function wrt S in Π
4. l be a literal in S
5. $p = \langle r_1, \ldots, r_n \rangle$ be a normal proof in $P(l, S, \Pi)$
6. r_i be a rule in p
7. l' be a literal in $body_+(r_i)$

Then:

1. $f(l) > f(l')$
2. $p_i = \langle r_1, \ldots, r_i \rangle$ is a normal proof of $h_S(r_i)$

Lemma 4.3.8. Let:

1. Π_1 and Π_2 be head-cycle-free A-Prolog programs with corresponding answer sets $S_1 \subset S_2$ (proper subset)
2. l be a literal in $S_2 \setminus S_1$
3. $p = \langle r_1, \ldots, r_n \rangle$ be a normal proof in $P(l, S_2, \Pi_2)$, so $h_{S_2}(r_n) = l$

Then there exists a rule $r \in p$ such that $r \in \Pi_2 \setminus \Pi_1$.

Proof. Let f be the ranking function wrt S_2 in Π_2. We employ induction on $f(l)$.

- Base step: $f(l) = \min\{f(l_0) : l_0 \in S_2 \setminus S_1\}$.
    1. To the contrary, assume: for every rule $r \in p$, we have $r \in \Pi_1 \cap \Pi_2$.
    2. Then $r_n \in \Pi_1$.
    3. Since p is a normal proof of l, for each literal $l' \in body_+(r_n)$, we have $f(l') < f(l) = \min\{f(l_0) : l_0 \in S_2 \setminus S_1\}$, so $l' \in S_1 \cap S_2$.
    4. Then r_n fires wrt S_1 (recall: r_n fires wrt S_2, and $S_1 \subset S_2$).
    5. As S_1 is an answer set of Π_1, we know S_1 satisfies $head(r_n)$.
    6. Let l' be a literal in $head(r_n)$.
        6.1. Case 1 of 2: $l' = l$. We already assumed $l \in S_2 \setminus S_1$.
        6.2. Case 2 of 2: $l' \neq l$. We have $l' \notin S_2$ (as only l is supported by r_n wrt S_2 in Π_2), so $l' \notin S_1$.

        In both cases, $l' \notin S_1$. So S_1 does not satisfy $head(r_n)$, contradiction.
- Induction step: $f(l) \leq \max\{f(l_0) : l_0 \in S_2 \setminus S_1\}$.

1. Induction hypothesis: for each literal $l' \in S_2 \setminus S_1$, let p' be a normal proof in $P(l', S_2, \Pi_2)$, if $f(l') < f(l)$, then there exists a rule $r \in p'$ such that $r \in \Pi_2 \setminus \Pi_1$.
2. To the contrary, assume for every rule $r \in p$, we have $r \in \Pi_1 \cap \Pi_2$.
   2.1. Case 1 of 2: there exists a literal $l' \in body_+(r_n)$ where $l' \in S_2 \setminus S_1$.
      2.1.1. Notice $f(l') < f(l) = h_{S_2}(r_n)$.
      2.1.2. Choose some positive integer $m < n$ where $h_{S_2}(r_m) = l'$.
      2.1.3. As $p = \langle r_1, \ldots, r_n \rangle$ is a normal proof of l, we know $p' = \langle r_1, \ldots, r_m \rangle$ is a normal proof of l', wrt S_2 in Π_2.
      2.1.4. By the induction hypothesis, p' contains some rule $r' \in \Pi_2 \setminus \Pi_1$.
      2.1.5. So p also contains r'.
      2.1.6. But we assumed $r \in \Pi_1 \cap \Pi_2$ for every rule $r \in p$, contradiction.
   2.2. Case 2 of 2: for every literal $l' \in body_+(r_n)$, we have $l' \in S_1 \cap S_2$.
      2.2.1. Then the rule r_n fires wrt S_1.
      2.2.2. By our assumption, $r_n \in \Pi_1$.
      2.2.3. As S_1 is an answer set of Π_1, we know S_1 satisfies $head(r_n)$.
      2.2.4. Let l' be a literal in $head(r_n)$.
         2.2.4.1. Subcase 1 of 2: $l' = l$. We already assumed $l \in S_2 \setminus S_1$.
         2.2.4.2. Subcase 2 of 2: $l' \neq l$. We have $l' \notin S_2$ (as only l is supported by r_n wrt S_2 in Π_2), so $l' \notin S_1$.

         In both subcases, $l' \notin S_1$. So S_1 does not satisfy $head(r_n)$, contradiction.

□

Remark 4.3.9. Let $p = \langle r_1, \ldots, r_n \rangle$ be a proof in $P(l, S, \Pi)$. If p is a minimal proof, then:

1. p is a normal proof
2. l depends on $h_S(r_i)$ for all $i < n$ (in the sense of Definition 4.1.6)

Lemma 4.3.10. Let Π be a nondisjunctive acyclic CR-Prolog program. If Π has answer sets $S_1 \subset S_2$, then there exist literals l_1 and l_2 in $head(\Pi^{cr})$ such that l_1 depends on l_2.

Proof. Some notations first:

1. By the contrapositive of Lemma 4.2.10, there exist abductive supports R_1 and R_2 (respectively corresponding to S_1 and S_2) where $head(R_1) \neq head(R_2)$.
2. Define two sets of facts: ${R_1}' = \phi(\alpha(R_1))$ and ${R_2}' = \phi(\alpha(R_2))$.
3. Introduce A-Prolog programs $\Pi_1 = \Pi^{reg} \cup {R_1}'$ and $\Pi_2 = \Pi^{reg} \cup {R_2}'$. By Lemma 4.2.9, S_1 and S_2 are respectively answer sets of Π_1 and Π_2.

We follow these steps:

1. Note that Π_1 and Π_2 are nondisjunctive. By the contrapositive of Lemma 4.2.6, the cr-literals appearing in R_1 are pairwise distinct. So are the cr-literals appearing in R_2. Then $|{R_1}'| = |R_1|$ and $|{R_2}'| = |R_2|$.
2. Notice $|R_1| = |R_2| > 0$. Then $|{R_1}'| = |{R_2}'| > 0$.
3. Observe $|head({R_1}')| = |{R_1}'|$ and $|head({R_2}')| = |{R_2}'|$. Thus $|head({R_1}')| = |head({R_2}')| > 0$.
4. Recall $head({R_1}') = head(R_1) \neq head(R_2) = head({R_2}')$. Then $head({R_1}') \setminus head({R_2}') \neq \varnothing$.

5. Select some literal $l_1 \in head(R_1{}') \setminus head(R_2{}')$. Let r_1 be the fact "l_1." in $R_1{}' \setminus R_2{}'$.

6. Since S_1 is an answer set of Π_1, we have $l_1 \in S_1$. Recall $S_1 \subset S_2$. Then $l_1 \in S_2$.

7. As S_2 is an answer set of Π_2, there exists a rule $r \in \Pi_2$ which supports l_1 wrt S_2. Note that $body_+(r) \subseteq S_2$.

   7.1. Case 1 of 2: there exists a literal $l \in body_+(r)$ where $l \in S_2 \setminus S_1$.

      7.1.1. Let p be a minimal proof in $P(l, S_2, \Pi_2)$.

      7.1.2. By Lemma 4.3.8, there exists a rule $r_2 \in p$ where $r_2 \in \Pi_2 \setminus \Pi_1$.

      7.1.3. Then $r_2 \in R_2{}' \setminus R_1{}' \subseteq \alpha(\Pi^{cr})$. Let literal $l_2 = h_{S_2}(r_2) \in head(\Pi^{cr})$.

      7.1.4. As p is a minimal proof, l depends on l_2.

      7.1.5. Recall l_1 depends on l in r. By transitivity, l_1 depends on l_2.

   7.2. Case 2 of 2: $body_+(r) \subseteq S_1 \subset S_2$. We show that this case is impossible.

      7.2.1. Subcase 1 of 2: $r \in \Pi_1 \cap \Pi_2$.

         7.2.1.1. Recall r supports l_1 wrt S_2. Since $body_+(r) \subseteq S_1 \subset S_2$, we know r also supports l_1 wrt S_1.

         7.2.1.2. Applying Lemma 4.2.7 to Π_1, we have $r = r_1$.

         7.2.1.3. However, $r \in \Pi_2$ whereas $r_1 \in \Pi_1 \setminus \Pi_2$, contradiction.

      7.2.2. Subcase 2 of 2: $r \in \Pi_2 \setminus \Pi_1$.

         7.2.2.1. So $r \in R_2{}' \setminus R_1{}'$. Then r is the fact "l_1.", which is exactly r_1.

         7.2.2.2. However, we selected r_1 from $R_1{}' \setminus R_2{}'$ while $r \in R_2{}'$, contradiction.

□

Remark 4.3.11. If a CR-Prolog program Π is acyclic, then there exists a nondisjunctive acyclic CR-Prolog program Π' equivalent to Π [Ben-Eliyahu and Dechter, 1994, page 73, Theorem 4.17].

Theorem 4.3.12 (Antichain Property of CR-Independent Acyclic CR-Prolog Programs)**.** Let Π be an acyclic CR-Prolog program. If Π is cr-independent, then Π has antichain property.

Proof. By Remark 4.3.11, there exists a nondisjunctive acyclic CR-Prolog Π' equivalent to Π. By the contrapositive of Lemma 4.3.10, Π' has antichain property. Therefore, Π has antichain property too. □

5 Conclusion

We have found a reasonably weak syntactic condition which guarantees that a CR-Prolog program has antichain property: the program being acyclic and cr-independent. The future goal is to further relax this sufficient condition to extend the class of CR-Prolog programs known to have antichain property.

6 Acknowledgment

We sincerely appreciate the significant guidance of Dr. Michael Gelfond. This paper includes his proofs of Theorem 3.1.3 and Lemmas 3.3.6 & 4.2.5. We also thank Evgenii Balai for the informative discussions.

References

Marcello Balduccini and Michael Gelfond. Logic programs with consistency-restoring rules. In *International Symposium on Logical Formalization of Commonsense Reasoning, AAAI 2003 Spring Symposium Series*, volume 102, 2003.

Rachel Ben-Eliyahu and Rina Dechter. Propositional semantics for disjunctive logic programs. *Annals of Mathematics and Artificial intelligence*, 12(1):53–87, 1994.

Esra Erdem, Michael Gelfond, and Nicola Leone. Applications of answer set programming. *AI Magazine*, 37 (3):53–68, 2016.

Michael Gelfond and Yulia Kahl. *Knowledge representation, reasoning, and the design of intelligent agents: The answer-set programming approach*. Cambridge University Press, 2014.

Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *ICLP/SLP*, volume 88, pages 1070–1080, 1988.